

Techniques for Generating Abstract Visual Animations with Cellular Automata

Frode Holm

CPU Productions, Santa Barbara, California, USA

info@cpuproductions.com

ABSTRACT

Several techniques are presented for exploring the infinite world of animated visuals made possible by cellular automata. Methods for creating and discovering new CA rules without using programming languages are discussed with examples.

CCS CONCEPTS

• **Computing methodologies** → Computer graphics.

KEYWORDS

Cellular Automata, abstract animations, generative art

ACM Reference Format:

Frode Holm. 2023. Techniques for Generating Abstract Visual Animations with Cellular Automata. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '23 Talks)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3587421.3595420>

1 INTRODUCTION

Cellular Automata is an established discipline in computer science, but has rarely been utilized to produce animations for the sake of art. This presentation will discuss some methods and techniques for creating CA rules that produce animations of remarkable complexity and beauty. Animations and figures are provided in an accompanying video.

A common way to describe a CA is as a 2-dimensional uniform grid composed of *Cells*. Each Cell has an associated *State* and some or all of the neighboring Cells' State is visible to it. On each tick of a global clock, the next State for the Cell is determined by some *Rule* logic that takes into account all or some of the information available to the Cell. There are many other ways to define a CA, but that is the one we will use here.

To compute the CA one can either run a program that embodies the Rule, for each Cell on each tick, or one can precompute the Rule and place the results in a lookup table. The latter is vastly more efficient but places some severe restrictions on the available data for a Cell, since the table must enumerate every possible permutation for that data. All of the following assumes that the CA is run with such a table. In addition to the new cell-value, each entry also contains the RGB-color for that new value, allowing for maximum flexibility in displaying the Rule.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Talks, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0143-6/23/08.

<https://doi.org/10.1145/3587421.3595420>

		8SUM								
STATE		0	< 3	< 6	< 9	< 12	< 15	< 18	< 21	<= 24
	0	0F	3C	2F	5E	1C	4E	3B	0A	5F
	1	3C	5C	0E	5E	1F	3E	3B	4B	3C
	2	5C	3E	4F	0B	1E	2B	5E	4F	2A
	3	0F	4B	4F	0A	3F	0A	0C	5A	0B

Figure 1: Parameters for a matrix-enabled Rule.

What becomes clear very soon when one starts to create CA Rules to make visuals, is that it is exceedingly hard to do. CA tend to behave in ways that are very difficult to predict using pure logic and reasoning, as you would be doing writing programs in a computer language.

To find truly unique behavior that evolves over long time spans, this author has found it necessary to employ what might be called “guided serendipity”. Thus, no mention will be given here on how to write source code to create Rules.

2 MATRIX PARAMETERIZATION

Parametrized rules have been employed successfully in the past [Rucker, 1989] and the following expands on this idea. The objective is to carve out a search space in which we know for sure that something of interest exists. Start with the source code for an existing rule, then choose two dimensions that each contains a measure of the available information. The requirement is only that the axes are not mutually exclusive, and each covers the entire range of possible values.

For example, one axis could be the Cell's public bits (State 0-3) and the other the sum of all the neighbors' public bits (8Sum 0-24). For each intersection in the matrix-grid, supply a value that will get picked up by the rule-generating algorithm and applied at strategic points to alter the rule-logic in small or big ways. The details of this procedure will, of course, be very specific to the rule being modified in this manner. A good rule of thumb is to create 4-5 variations of the rule, each differing from the others in significant ways, then, based on the parameters, switch between them according to the state of the Cells. Then, through trial and error, explore this space for new, interesting behavior.

In this example, the numbers denote branches in the Rule-logic and the letters have a similar function in assigning color values.

3 MERGE RULES

“Merge Rules” has proven to be a very powerful tool to discover new rules. In distinction to what was described above, this method does not require writing any new code. All that is needed are several existing Rule lookup tables.

The idea for Merge is quite simple. Take two Rules, say, and run them for a fair number of clock ticks. While doing so, record and

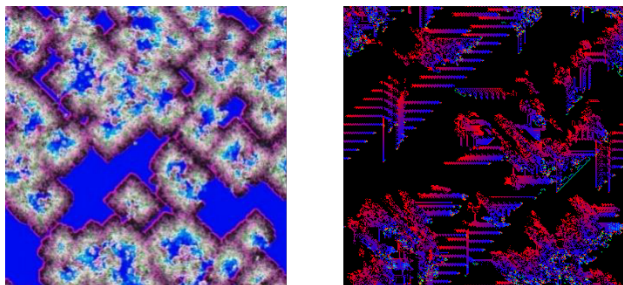


Figure 2: Two different merges of four unrelated Rules. Scenes 7a and 7b in the video.

add up each entry in the lookup table that is visited. After such a run, create a new empty Rule and then compare the statistics for the Rules in the Merge. For each entry in the table, compare the number of ‘hits’ generated by each Rule. Whichever has the most, gets that entry copied into the new Rule.

This simple procedure has generated many truly amazing Rules. Sometimes one gets a combination of the behaviors of the participating Rules, sometimes a totally unexpected result.

4 EVOLUTION

One idea the author has been pursuing for discovering new Rules, has been to borrow a page from the field of Artificial Life and to try to place Rules in an evolutionary process. The objective here is to somehow automate the search through the vast Rule space.

As with most AL schemes for evolution, a Rule is divided into Genes which are exchanged between two ‘parents’, including random ‘mutations’, and which then produces a ‘child’ Rule. This Rule is then evaluated for ‘fitness’ using one or more fitness functions and is then ranked among all children of that generation. Only the top performers are subsequently promoted to repeat the procedure over again.

For now, the two main problems to overcome are first of all the lack of original Rules to serve as ‘ancestors’. As any student of evolutionary statistics, biological or digital, will tell us, is that one needs a large population to avoid inbreeding. In our case, that means Rules produced by independent source code and not by any of the methods discussed above, since they will already have ‘genetic’ material in common. It is hoped that this issue can be solved by crowd-sourcing the creation of new Rules and a website is being set up for that purpose.

The second problem is creating good fitness functions. How do you measure what is an ‘interesting’ Rule? The author has no good answer to that question but has instead relied on several well-known signal processing techniques and math measurements. The following are some functions that have proven useful in pushing the evolutionary process in a fruitful direction.

- *Similarity.* This is a measure of continuity between one frame and the next.
- *Edge detection.* Do the generated images have any detectable continuous edges?
- *Color regions.* Does the image have significant regions of similar or related colors?

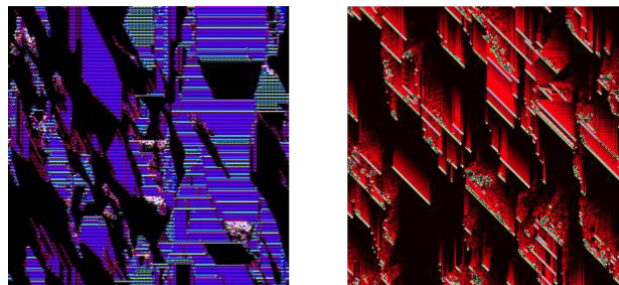


Figure 3: Rules produced by Evolution. Scenes 8a and 8b in the video.

- *Randomness.* This measure is performed in both space and time.

Despite all these difficulties ahead, Evolution is a very promising approach that warrants further study and development.

5 CONCLUSION

With the techniques described above the author has discovered a wealth of unexpected and fascinating CA visual behavior. Other promising methods include ‘composite’ Rules, where two or more Rules are interleaved in time and/or space. Even though CA has been around for a long time, the author is continually astonished by the never-seen-before (to him) emerging visuals that defies expectations and comprehension.

REFERENCES

- Kamalika Bhattacharjee Et. Al. 2018. A Survey of Cellular Automata: Types, Dynamics, Non-uniformity and Applications. Springer Link.
 Rudy Rucker. 1989. CA Lab, Exploring Cellular Automata. Autodesk.
 Tommaso Toffoli. and Norman Margolus. 1987. Cellular Automata Machines. MIT Press.